520- Project

Ritika Lama

Dataset

```r
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.5
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
gaming <- read_csv("gaming_data.csv")
```

```
Rows: 3024 Columns: 13
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (7): UserID, Gender, Country, Device, GameGenre, SpendingSegment, Payme...
dbl  (5): Age, SessionCount, AverageSessionLength, InAppPurchaseAmount, Firs...
date (1): LastPurchaseDate

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
names(gaming)
```

```
 [1] "UserID"                        "Age"
 [3] "Gender"                        "Country"
 [5] "Device"                        "GameGenre"
 [7] "SessionCount"                  "AverageSessionLength"
 [9] "SpendingSegment"               "InAppPurchaseAmount"
[11] "FirstPurchaseDaysAfterInstall" "PaymentMethod"
[13] "LastPurchaseDate"
```

Clean the dataset and summarise

```r
gaming_clean <- gaming %>%
  mutate(
    LastPurchaseDate = suppressWarnings(ymd(LastPurchaseDate)),
    InAppPurchaseAmount = if_else(is.na(InAppPurchaseAmount), 0, InAppPurchaseAmount),
    PaymentMethod = if_else(is.na(PaymentMethod) & InAppPurchaseAmount == 0, "No Purchase", I
    Age = if_else(is.na(Age), median(Age, na.rm = TRUE), Age),
    Gender = replace_na(Gender, "Unknown"),
    Country = replace_na(Country, "Unknown"),
    Device = replace_na(Device, "Unknown"),
    GameGenre = replace_na(GameGenre, "Unknown")
  )

max_purchase_date <- max(gaming_clean$LastPurchaseDate, na.rm = TRUE)

gaming_clean <- gaming_clean %>%
  mutate(
    RecencyDays = as.numeric(max_purchase_date - LastPurchaseDate),
    TotalPlaytime = SessionCount * AverageSessionLength,
    LogSpend = log1p(InAppPurchaseAmount),
    IsPayer = if_else(InAppPurchaseAmount > 0, 1, 0)
  )

head(gaming_clean)
```

```
# A tibble: 6 x 17
  UserID   Age Gender Country Device GameGenre SessionCount AverageSessionLength
  <chr>  <dbl> <chr>  <chr>   <chr>  <chr>            <dbl>                 <dbl>
```

```
1 c9889~    49 Male   Norway  Andro~ Battle R~            9                12.8
2 7c9e4~    15 Male   Switze~ iOS    Action R~           11                19.4
3 fd61e~    23 Male   China   Andro~ Fighting             9                 8.87
4 bdb7f~    31 Male   Mexico  Andro~ Racing              12                19.6
5 aa7ee~    37 Female India   Andro~ Battle R~           10                15.2
6 99223~    38 Female China   Andro~ Fighting            16                26.0
# i 9 more variables: SpendingSegment <chr>, InAppPurchaseAmount <dbl>,
#   FirstPurchaseDaysAfterInstall <dbl>, PaymentMethod <chr>,
#   LastPurchaseDate <date>, RecencyDays <dbl>, TotalPlaytime <dbl>,
#   LogSpend <dbl>, IsPayer <dbl>
```

```r
str(gaming_clean)
```

```
tibble [3,024 x 17] (S3: tbl_df/tbl/data.frame)
 $ UserID                      : chr [1:3024] "c9889ab0-9cfc-4a75-acd9-5eab1df0015c" "7c9e4
 $ Age                         : num [1:3024] 49 15 23 31 37 38 20 34 38 46 ...
 $ Gender                      : chr [1:3024] "Male" "Male" "Male" "Male" ...
 $ Country                     : chr [1:3024] "Norway" "Switzerland" "China" "Mexico" ...
 $ Device                      : chr [1:3024] "Android" "iOS" "Android" "Android" ...
 $ GameGenre                   : chr [1:3024] "Battle Royale" "Action RPG" "Fighting" "Racin
 $ SessionCount                : num [1:3024] 9 11 9 12 10 16 9 15 10 18 ...
 $ AverageSessionLength        : num [1:3024] 12.83 19.39 8.87 19.56 15.23 ...
 $ SpendingSegment             : chr [1:3024] "Minnow" "Minnow" "Minnow" "Minnow" ...
 $ InAppPurchaseAmount         : num [1:3024] 11.4 6.37 15.81 13.49 10.86 ...
 $ FirstPurchaseDaysAfterInstall: num [1:3024] 28 18 30 9 15 28 24 30 1 19 ...
 $ PaymentMethod               : chr [1:3024] "Apple Pay" "Debit Card" "Apple Pay" "Debit Ca
 $ LastPurchaseDate            : Date[1:3024], format: "2025-03-19" "2025-06-08" ...
 $ RecencyDays                 : num [1:3024] 147 66 72 134 100 100 70 109 127 191 ...
 $ TotalPlaytime               : num [1:3024] 115.5 213.3 79.8 234.7 152.3 ...
 $ LogSpend                    : num [1:3024] 2.52 2 2.82 2.67 2.47 ...
 $ IsPayer                     : num [1:3024] 1 1 1 1 1 1 1 1 1 1 ...
```

```r
names(gaming_clean)
```

```
 [1] "UserID"                       "Age"
 [3] "Gender"                       "Country"
 [5] "Device"                       "GameGenre"
 [7] "SessionCount"                 "AverageSessionLength"
 [9] "SpendingSegment"              "InAppPurchaseAmount"
[11] "FirstPurchaseDaysAfterInstall" "PaymentMethod"
[13] "LastPurchaseDate"             "RecencyDays"
```

```
[15] "TotalPlaytime"                 "LogSpend"
[17] "IsPayer"
```

```r
colSums(is.na(gaming_clean))
```

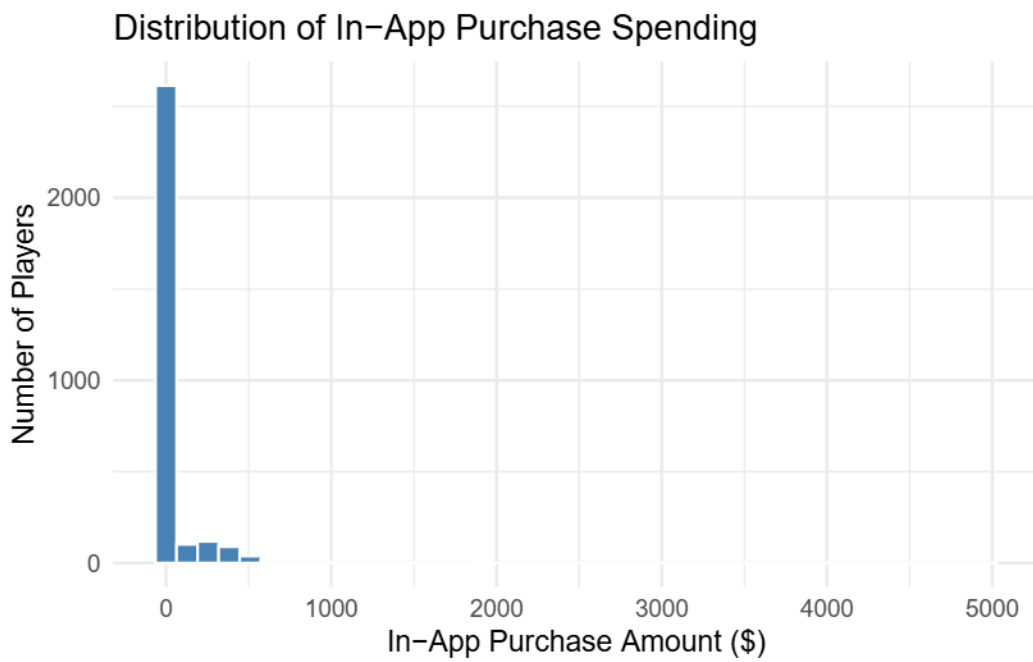
```
                       UserID                          Age
                            0                            0
                       Gender                      Country
                            0                            0
                       Device                    GameGenre
                            0                            0
                 SessionCount          AverageSessionLength
                            0                            0
              SpendingSegment           InAppPurchaseAmount
                            0                            0
FirstPurchaseDaysAfterInstall                 PaymentMethod
                          136                            0
             LastPurchaseDate                   RecencyDays
                          136                          136
                TotalPlaytime                      LogSpend
                            0                            0
                      IsPayer
                            0
```

```r
summary(gaming_clean)
```

```
   UserID               Age           Gender            Country
 Length:3024        Min.   :13.00   Length:3024        Length:3024
 Class :character   1st Qu.:24.00   Class :character   Class :character
 Mode  :character   Median :33.00   Mode  :character   Mode  :character
                    Mean   :33.52
                    3rd Qu.:44.00
                    Max.   :54.00

   Device           GameGenre          SessionCount   AverageSessionLength
 Length:3024        Length:3024        Min.   : 1.00   Min.   : 5.01
 Class :character   Class :character   1st Qu.: 8.00   1st Qu.:12.68
 Mode  :character   Mode  :character   Median :10.00   Median :20.32
                                       Mean   :10.07   Mean   :20.07
                                       3rd Qu.:12.00   3rd Qu.:27.42
                                       Max.   :22.00   Max.   :34.99
```

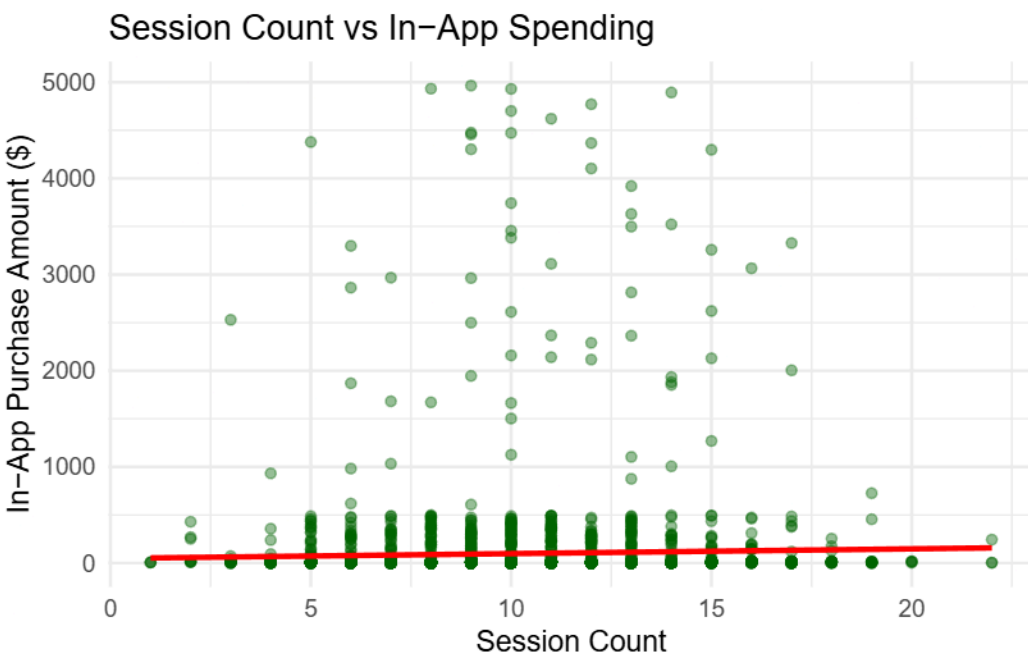
```
 SpendingSegment    InAppPurchaseAmount FirstPurchaseDaysAfterInstall
 Length:3024        Min.   :   0.00     Min.   : 0.00
 Class :character   1st Qu.:   5.12     1st Qu.: 8.00
 Mode  :character   Median :  11.46     Median :16.00
                    Mean   :  97.97     Mean   :15.38
                    3rd Qu.:  17.55     3rd Qu.:23.00
                    Max.   :4964.45     Max.   :30.00
                                        NA's   :136
 PaymentMethod      LastPurchaseDate      RecencyDays    TotalPlaytime
 Length:3024        Min.   :2025-01-01   Min.   :  0.0   Min.   : 11.01
 Class :character   1st Qu.:2025-02-28   1st Qu.: 54.0   1st Qu.:115.92
 Mode  :character   Median :2025-04-26   Median :109.0   Median :186.38
                    Mean   :2025-04-24   Mean   :110.3   Mean   :201.96
                    3rd Qu.:2025-06-20   3rd Qu.:166.0   3rd Qu.:273.21
                    Max.   :2025-08-13   Max.   :224.0   Max.   :732.16
                    NA's   :136          NA's   :136
   LogSpend        IsPayer
 Min.   :0.000   Min.   :0.0000
 1st Qu.:1.812   1st Qu.:1.0000
 Median :2.522   Median :1.0000
 Mean   :2.614   Mean   :0.9547
 3rd Qu.:2.921   3rd Qu.:1.0000
 Max.   :8.510   Max.   :1.0000
```

```r
# ----------------------------
# Graph 1: Distribution of Spending
# ----------------------------
ggplot(gaming_clean, aes(x = InAppPurchaseAmount)) +
  geom_histogram(bins = 40, fill = "steelblue", color = "white") +
  labs(
    title = "Distribution of In-App Purchase Spending",
    x = "In-App Purchase Amount ($)",
    y = "Number of Players"
  ) +
  theme_minimal()
```



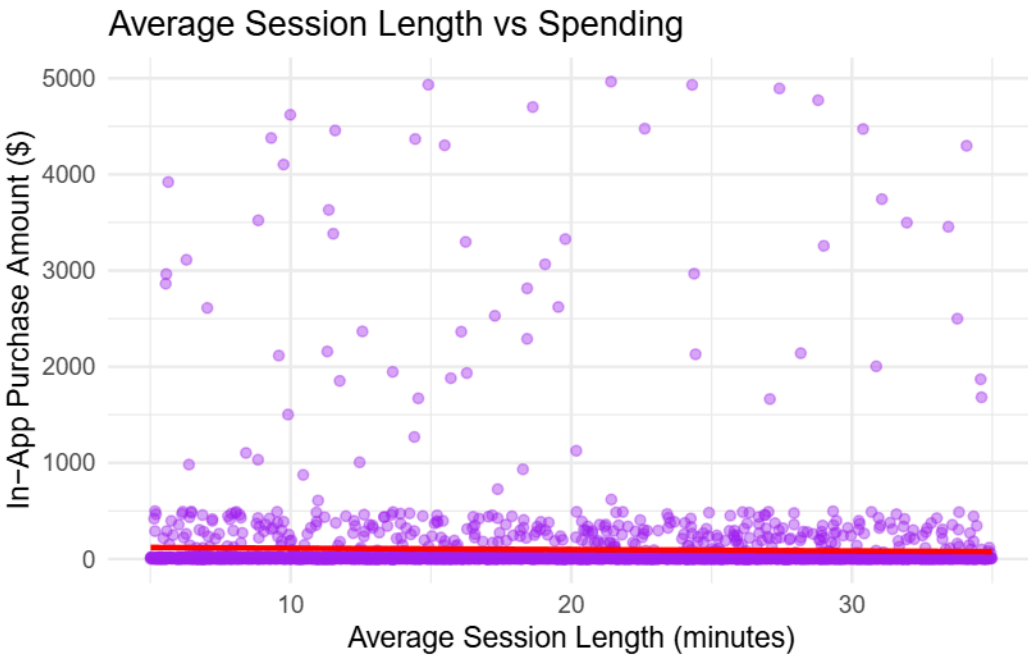

```
# ----------------------------
# Graph 2: Session Count vs Spending
# ----------------------------
ggplot(gaming_clean, aes(x = SessionCount, y = InAppPurchaseAmount)) +
  geom_point(alpha = 0.4, color = "darkgreen") +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(
    title = "Session Count vs In-App Spending",
    x = "Session Count",
    y = "In-App Purchase Amount ($)"
  ) +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```

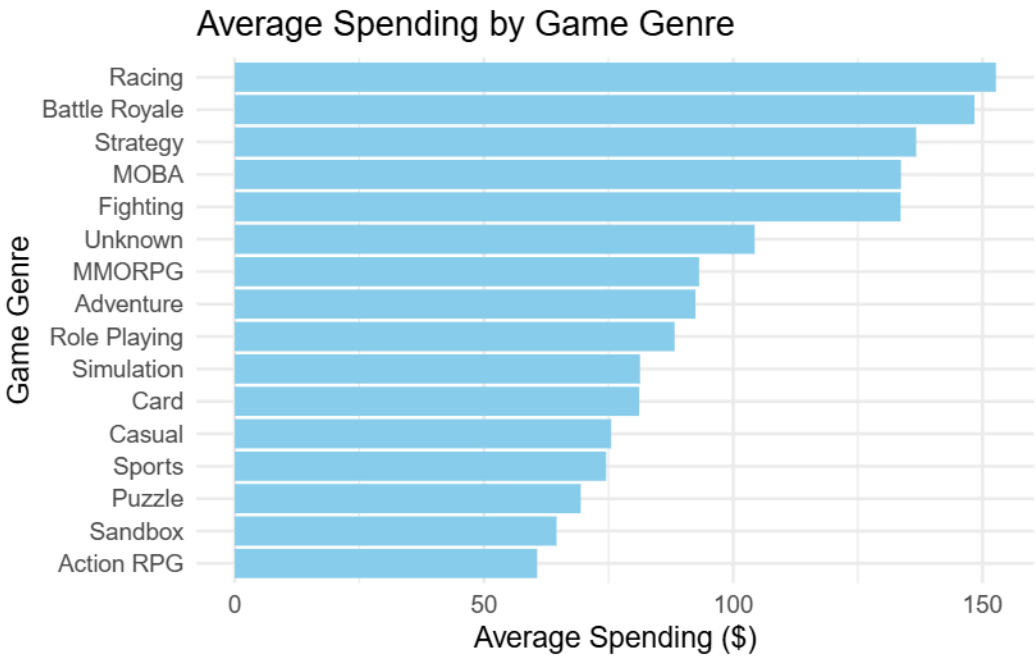



```r
# ----------------------------
# Graph 3: Average Session Length vs Spending
# ----------------------------
ggplot(gaming_clean, aes(x = AverageSessionLength, y = InAppPurchaseAmount)) +
  geom_point(alpha = 0.4, color = "purple") +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(
    title = "Average Session Length vs Spending",
    x = "Average Session Length (minutes)",
    y = "In-App Purchase Amount ($)"
  ) +
  theme_minimal()
```

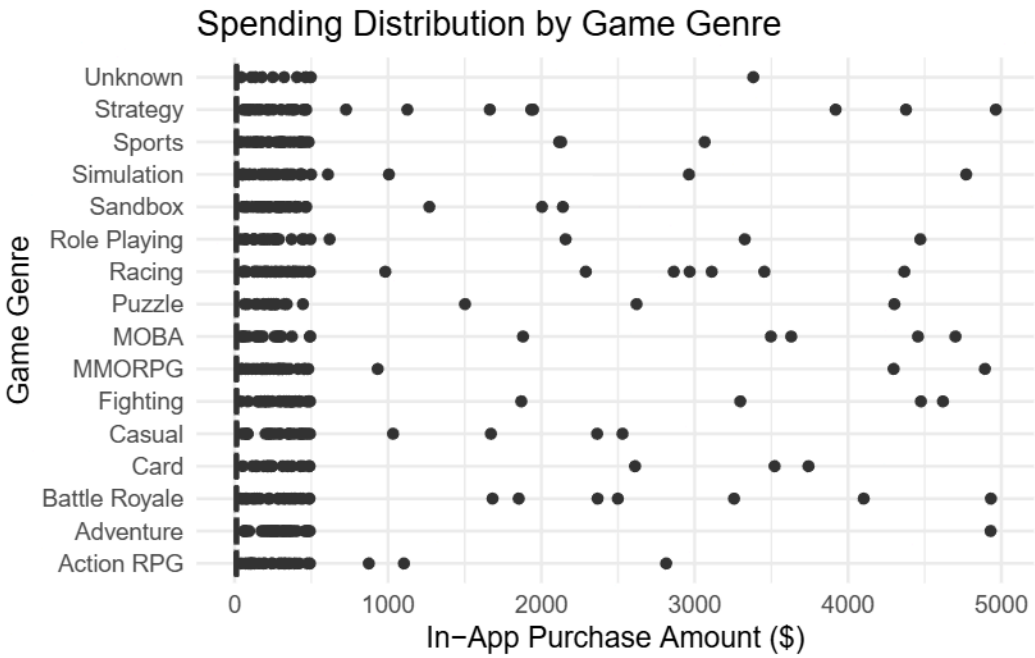
```
`geom_smooth()` using formula = 'y ~ x'
```




```r
# ----------------------------
# Graph 4: Average Spending by Game Genre
# ----------------------------
gaming_clean %>%
  group_by(GameGenre) %>%
  summarise(AverageSpending = mean(InAppPurchaseAmount, na.rm = TRUE)) %>%
  arrange(desc(AverageSpending)) %>%
  ggplot(aes(x = reorder(GameGenre, AverageSpending), y = AverageSpending)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  coord_flip() +
  labs(
    title = "Average Spending by Game Genre",
    x = "Game Genre",
    y = "Average Spending ($)"
  ) +
  theme_minimal()
```




```r
# ----------------------------
# Graph 5: Boxplot of Spending by Genre
# ----------------------------
ggplot(gaming_clean, aes(x = GameGenre, y = InAppPurchaseAmount)) +
  geom_boxplot(fill = "lightgreen") +
  coord_flip() +
  labs(
    title = "Spending Distribution by Game Genre",
    x = "Game Genre",
    y = "In-App Purchase Amount ($)"
  ) +
  theme_minimal()
```




```
# --------------------------
# Modeling dataset
# --------------------------
model_df <- gaming_clean %>%
  select(
    LogSpend, InAppPurchaseAmount, Age, Gender, Country, Device, GameGenre,
    SessionCount, AverageSessionLength, FirstPurchaseDaysAfterInstall,
    PaymentMethod, RecencyDays, TotalPlaytime
  ) %>%
  drop_na(LogSpend, Age, Gender, Country, Device, GameGenre,
          SessionCount, AverageSessionLength, PaymentMethod, RecencyDays, TotalPlaytime)

#Split training and testing
set.seed(42)
train_index <- sample(seq_len(nrow(model_df)), size = 0.8 * nrow(model_df))
train <- model_df[train_index, ]
test <- model_df[-train_index, ]
```

K-means Player Segmentation (Minnow, Dolphin, Whale)

```r
set.seed(42)
# Use payer + engagement features for clustering
cluster_df <- gaming_clean %>%
  select(UserID, InAppPurchaseAmount, SessionCount, AverageSessionLength, TotalPlaytime) %>%
  drop_na() %>%
  filter(InAppPurchaseAmount > 0)

cluster_scaled <- cluster_df %>%
  select(InAppPurchaseAmount, SessionCount, AverageSessionLength, TotalPlaytime) %>%
  scale()

kmeans_model <- kmeans(cluster_scaled, centers = 3, nstart = 30)

cluster_results <- cluster_df %>%
  mutate(cluster = kmeans_model$cluster) %>%
  group_by(cluster) %>%
  mutate(cluster_avg_spend = mean(InAppPurchaseAmount, na.rm = TRUE)) %>%
  ungroup()

# Rank clusters by average spending and label as Minnow/Dolphin/Whale
cluster_rank <- cluster_results %>%
  distinct(cluster, cluster_avg_spend) %>%
  arrange(cluster_avg_spend) %>%
  mutate(PlayerType = c("Minnow", "Dolphin", "Whale"))

cluster_results <- cluster_results %>%
  left_join(cluster_rank, by = c("cluster", "cluster_avg_spend"))

cluster_rank
```

```
# A tibble: 3 x 3
  cluster cluster_avg_spend PlayerType
    <int>             <dbl> <chr>
1       1              43.6 Minnow
2       3              52.9 Dolphin
3       2            3308.  Whale
```

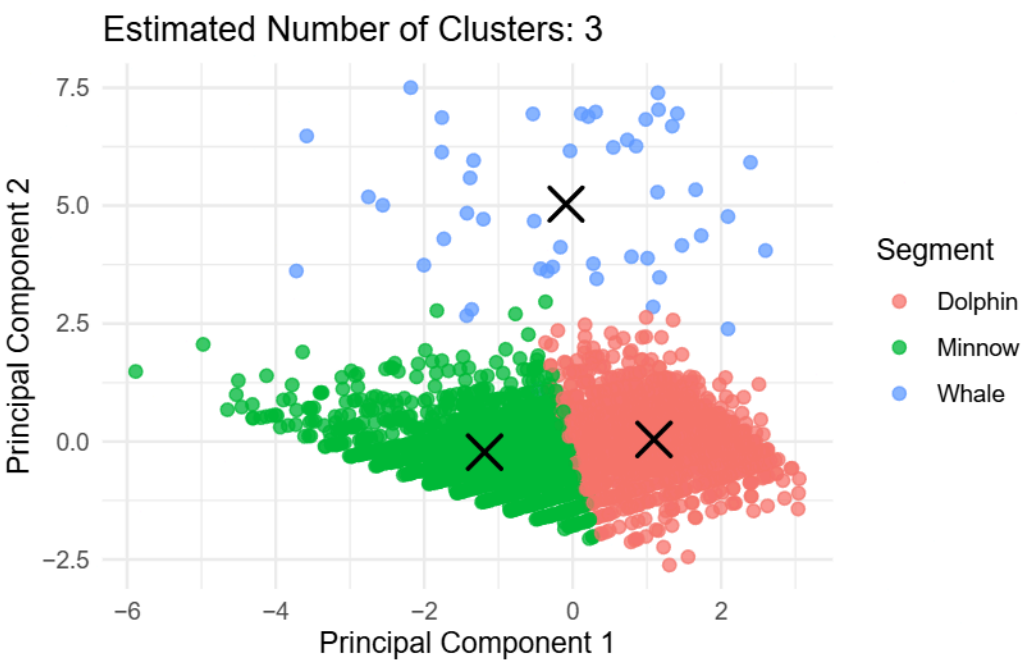
```r
cluster_results %>%
  group_by(PlayerType) %>%
  summarise(
    Players = n(),
```

```r
    AvgSpend = mean(InAppPurchaseAmount, na.rm = TRUE),
    AvgSessionCount = mean(SessionCount, na.rm = TRUE),
    AvgSessionLength = mean(AverageSessionLength, na.rm = TRUE),
    AvgTotalPlaytime = mean(TotalPlaytime, na.rm = TRUE)
  ) %>%
  arrange(match(PlayerType, c("Minnow", "Dolphin", "Whale")))
```

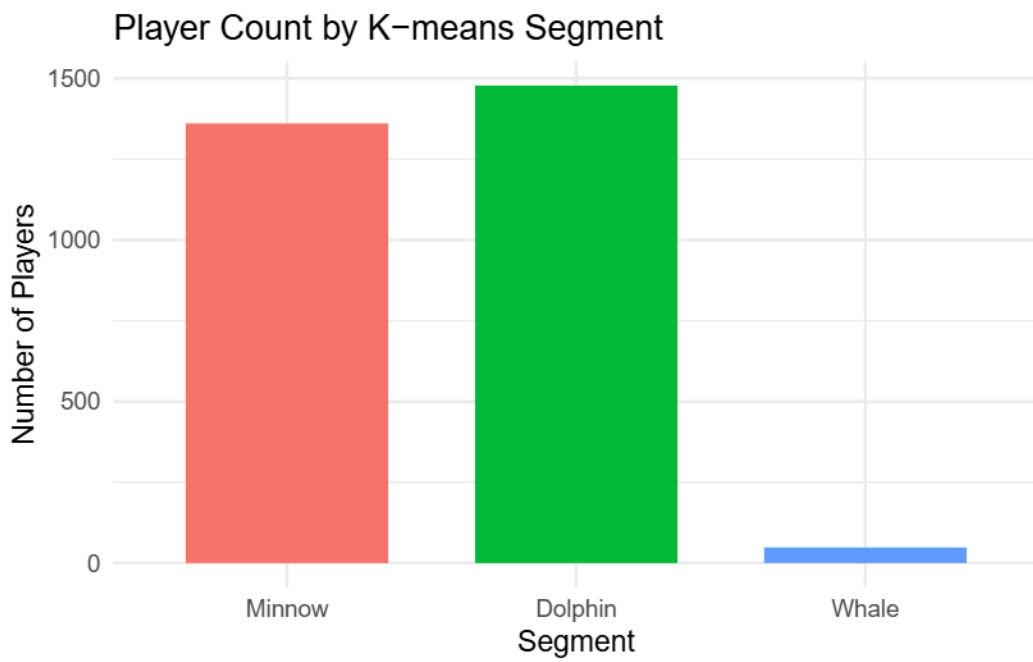
```
# A tibble: 3 x 6
  PlayerType Players AvgSpend AvgSessionCount AvgSessionLength AvgTotalPlaytime
  <chr>        <int>    <dbl>           <dbl>             <dbl>            <dbl>
1 Minnow        1361     43.6           11.1               26.9             293.
2 Dolphin       1478     52.9            9.05              13.8             118.
3 Whale           48   3308.            11.1               18.6             210.
```

```r
# 2D cluster map using PCA (similar to classic K-means cluster visualization)
pca_fit <- prcomp(cluster_scaled)

cluster_plot_df <- as_tibble(pca_fit$x[, 1:2]) %>%
  setNames(c("PC1", "PC2")) %>%
  mutate(PlayerType = cluster_results$PlayerType)

cluster_centers <- cluster_plot_df %>%
  group_by(PlayerType) %>%
  summarise(PC1 = mean(PC1), PC2 = mean(PC2), .groups = "drop")

ggplot(cluster_plot_df, aes(x = PC1, y = PC2, color = PlayerType)) +
  geom_point(alpha = 0.75, size = 2) +
  geom_point(
    data = cluster_centers,
    aes(x = PC1, y = PC2),
    color = "black",
    size = 5,
    shape = 4,
    stroke = 1.2,
    inherit.aes = FALSE
  ) +
  labs(
    title = "Estimated Number of Clusters: 3",
    x = "Principal Component 1",
    y = "Principal Component 2",
    color = "Segment"
```

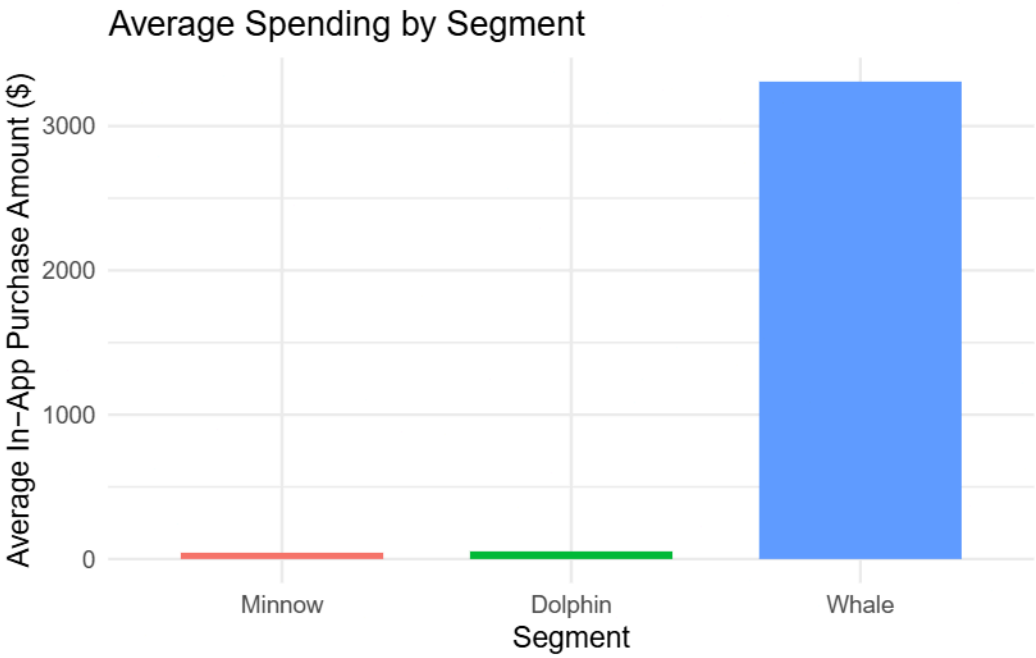
```
  ) +
  theme_minimal()
```



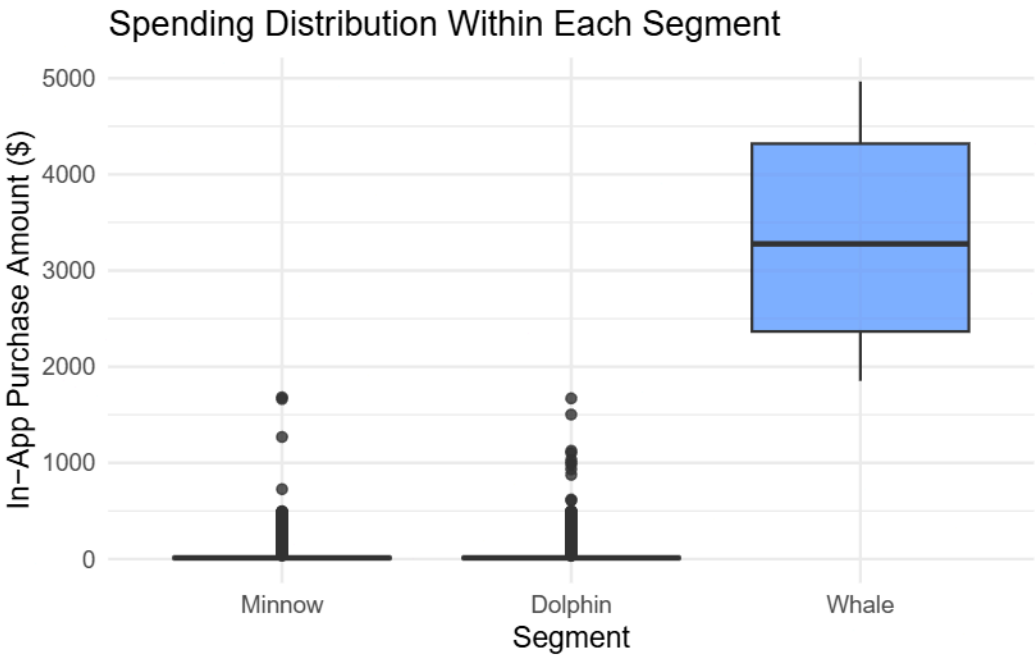

```
# Cluster sizes (how many players in each segment)
cluster_results %>%
  count(PlayerType) %>%
  mutate(PlayerType = factor(PlayerType, levels = c("Minnow", "Dolphin", "Whale"))) %>%
  ggplot(aes(x = PlayerType, y = n, fill = PlayerType)) +
  geom_col(width = 0.7, show.legend = FALSE) +
  labs(
    title = "Player Count by K-means Segment",
    x = "Segment",
    y = "Number of Players"
  ) +
  theme_minimal()
```




```r
# Segment spending profile
cluster_results %>%
  group_by(PlayerType) %>%
  summarise(AvgSpend = mean(InAppPurchaseAmount, na.rm = TRUE)) %>%
  mutate(PlayerType = factor(PlayerType, levels = c("Minnow", "Dolphin", "Whale"))) %>%
  ggplot(aes(x = PlayerType, y = AvgSpend, fill = PlayerType)) +
  geom_col(width = 0.7, show.legend = FALSE) +
  labs(
    title = "Average Spending by Segment",
    x = "Segment",
    y = "Average In-App Purchase Amount ($)"
  ) +
  theme_minimal()
```




```r
# Distribution of spending within each cluster
cluster_results %>%
  mutate(PlayerType = factor(PlayerType, levels = c("Minnow", "Dolphin", "Whale"))) %>%
  ggplot(aes(x = PlayerType, y = InAppPurchaseAmount, fill = PlayerType)) +
  geom_boxplot(show.legend = FALSE, alpha = 0.8) +
  labs(
    title = "Spending Distribution Within Each Segment",
    x = "Segment",
    y = "In-App Purchase Amount ($)"
  ) +
  theme_minimal()
```



Logistic Model: High vs Low Spender

After segmentation, we model whether a player is a high spender or not using behavior-related predictors.

```
# Binary target: high spender if spending >= 75th percentile
threshold <- quantile(gaming_clean$InAppPurchaseAmount, 0.75, na.rm = TRUE)

class_df <- gaming_clean %>%
  mutate(
    HighSpender = if_else(InAppPurchaseAmount >= threshold, 1, 0)
  ) %>%
  select(
    HighSpender, Age, SessionCount, AverageSessionLength,
    FirstPurchaseDaysAfterInstall, RecencyDays, TotalPlaytime
  ) %>%
  drop_na()

set.seed(42)
train_index_cls <- sample(seq_len(nrow(class_df)), size = 0.8 * nrow(class_df))
train_cls <- class_df[train_index_cls, ]
test_cls <- class_df[-train_index_cls, ]
```

```r
# Full logistic model with behavior variables
glm_full <- glm(
  HighSpender ~ Age + SessionCount + AverageSessionLength +
    FirstPurchaseDaysAfterInstall + RecencyDays + TotalPlaytime,
  data = train_cls,
  family = binomial()
)

glm_summary <- summary(glm_full)
glm_summary
```

```
Call:
glm(formula = HighSpender ~ Age + SessionCount + AverageSessionLength +
    FirstPurchaseDaysAfterInstall + RecencyDays + TotalPlaytime,
    family = binomial(), data = train_cls)

Coefficients:
                                Estimate Std. Error z value Pr(>|z|)
(Intercept)                   -8.407e-01  4.353e-01  -1.931   0.0534 .
Age                           -3.895e-03  3.980e-03  -0.979   0.3277
SessionCount                  -7.183e-03  3.825e-02  -0.188   0.8510
AverageSessionLength          -5.735e-04  1.817e-02  -0.032   0.9748
FirstPurchaseDaysAfterInstall  4.638e-03  5.317e-03   0.872   0.3830
RecencyDays                   -2.968e-04  7.300e-04  -0.407   0.6844
TotalPlaytime                  1.813e-05  1.740e-03   0.010   0.9917
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2679.2  on 2309  degrees of freedom
Residual deviance: 2677.1  on 2303  degrees of freedom
AIC: 2691.1

Number of Fisher Scoring iterations: 4
```

```r
# Keep only significant variables (p < 0.05); fallback if none
coefs <- coef(glm_summary)
sig_terms <- rownames(coefs)[coefs[, "Pr(>|z|)"] < 0.05]
sig_terms <- setdiff(sig_terms, "(Intercept)")
```

```r
if (length(sig_terms) == 0) {
  sig_terms <- c("SessionCount", "TotalPlaytime")
}

glm_formula <- as.formula(
  paste("HighSpender ~", paste(sig_terms, collapse = " + "))
)

glm_sig <- glm(glm_formula, data = train_cls, family = binomial())
summary(glm_sig)
```

```
Call:
glm(formula = glm_formula, family = binomial(), data = train_cls)

Coefficients:
                Estimate Std. Error z value Pr(>|z|)
(Intercept)   -9.373e-01  1.587e-01  -5.907 3.49e-09 ***
SessionCount  -7.397e-03  1.835e-02  -0.403    0.687
TotalPlaytime  2.153e-08  5.239e-04   0.000    1.000
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2679.2  on 2309  degrees of freedom
Residual deviance: 2679.0  on 2307  degrees of freedom
AIC: 2685

Number of Fisher Scoring iterations: 4
```

```r
# Classification performance on test set
glm_prob <- predict(glm_sig, newdata = test_cls, type = "response")
glm_pred <- if_else(glm_prob >= 0.5, 1, 0)

glm_cm <- table(Predicted = glm_pred, Actual = test_cls$HighSpender)
glm_cm
```

```
         Actual
Predicted   0   1
        0 438 140
```

```r
glm_accuracy <- mean(glm_pred == test_cls$HighSpender)
glm_accuracy
```

```
[1] 0.7577855
```

Random Forest Classification (High vs Low Spender)

To compare with a non-linear model, we fit a random forest and inspect which variables contribute most to classification.

```r
library(randomForest)
```

```
Warning: package 'randomForest' was built under R version 4.5.3

randomForest 4.7-1.2

Type rfNews() to see new features/changes/bug fixes.


Attaching package: 'randomForest'

The following object is masked from 'package:dplyr':

    combine

The following object is masked from 'package:ggplot2':

    margin
```

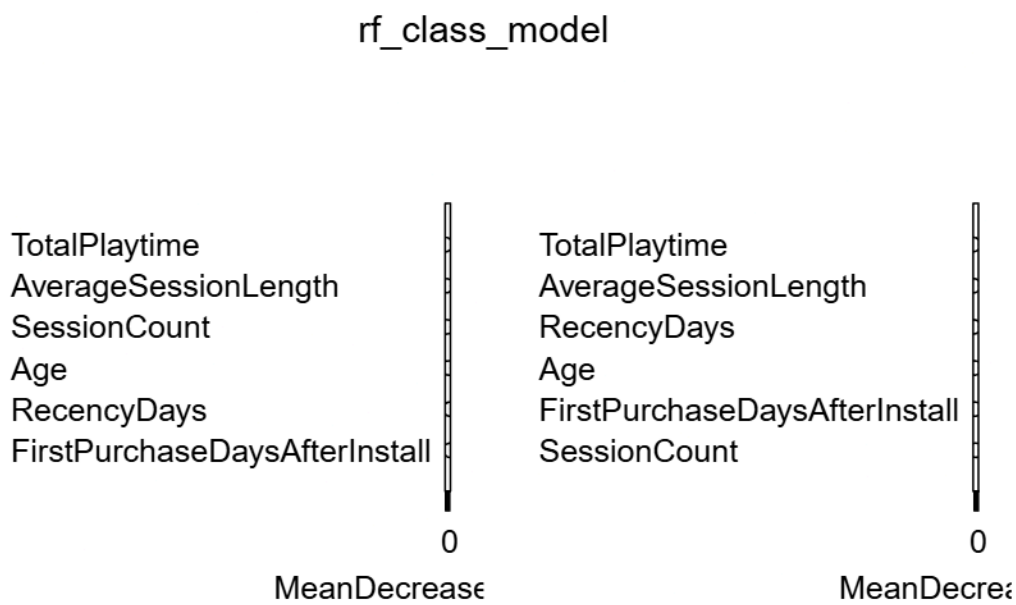
```r
set.seed(42)
rf_class_model <- randomForest(
  as.factor(HighSpender) ~ Age + SessionCount + AverageSessionLength +
    FirstPurchaseDaysAfterInstall + RecencyDays + TotalPlaytime,
  data = train_cls,
  ntree = 500,
  importance = TRUE
)

rf_class_model
```

```
Call:
 randomForest(formula = as.factor(HighSpender) ~ Age + SessionCount +      AverageSessionLen
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 28.61%
Confusion matrix:
     0  1 class.error
0 1634 60  0.03541913
1  601 15  0.97564935
```

```r
importance(rf_class_model)
```

```
                                         0          1 MeanDecreaseAccuracy
Age                             2.4305540   3.092935             3.631340
SessionCount                   16.9099673 -13.879871            10.647306
AverageSessionLength           20.8756613 -18.639331            14.146081
FirstPurchaseDaysAfterInstall -0.4325055  -3.925779            -2.493835
RecencyDays                    -1.4784111  -1.592249            -2.076796
TotalPlaytime                  22.2493915 -19.040192            16.339759
                               MeanDecreaseGini
Age                                    137.6455
SessionCount                            89.4639
AverageSessionLength                   184.2841
FirstPurchaseDaysAfterInstall          127.8535
RecencyDays                            176.3470
TotalPlaytime                          186.3397
```

```r
varImpPlot(rf_class_model)
```



The following test results show classification accuracy and confusion matrix for random forest.

```
# Random forest test performance
rf_pred <- predict(rf_class_model, newdata = test_cls)
rf_cm <- table(Predicted = rf_pred, Actual = as.factor(test_cls$HighSpender))
rf_cm
```

```
         Actual
Predicted   0   1
        0 421 132
        1  17   8
```

```
rf_accuracy <- mean(rf_pred == as.factor(test_cls$HighSpender))
rf_accuracy
```

```
[1] 0.7422145
```

Model Comparison (Logistic vs Random Forest)

This final table places both models side by side using the same metrics (Accuracy, Precision, Recall, F1).

```r
# Convert predictions to factors with consistent levels
actual_factor <- factor(test_cls$HighSpender, levels = c(0, 1))
glm_factor <- factor(glm_pred, levels = c(0, 1))
rf_factor <- factor(as.numeric(as.character(rf_pred)), levels = c(0, 1))

# Helper function for binary classification metrics
calc_metrics <- function(pred, actual) {
  cm <- table(Predicted = pred, Actual = actual)

  tp <- ifelse(!is.na(cm["1", "1"]), cm["1", "1"], 0)
  tn <- ifelse(!is.na(cm["0", "0"]), cm["0", "0"], 0)
  fp <- ifelse(!is.na(cm["1", "0"]), cm["1", "0"], 0)
  fn <- ifelse(!is.na(cm["0", "1"]), cm["0", "1"], 0)

  accuracy <- (tp + tn) / sum(cm)
  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))
  f1 <- ifelse((precision + recall) == 0, 0, 2 * precision * recall / (precision + recall))

  tibble(
    Accuracy = round(accuracy, 3),
    Precision = round(precision, 3),
    Recall = round(recall, 3),
    F1 = round(f1, 3)
  )
}

comparison_table <- bind_rows(
  calc_metrics(glm_factor, actual_factor) %>% mutate(Model = "Logistic Regression"),
  calc_metrics(rf_factor, actual_factor) %>% mutate(Model = "Random Forest")
) %>%
  select(Model, Accuracy, Precision, Recall, F1)

comparison_table
```

```
# A tibble: 2 x 5
  Model               Accuracy Precision Recall    F1
  <chr>                  <dbl>     <dbl>  <dbl> <dbl>
1 Logistic Regression    0.758      0      0     0
2 Random Forest          0.742      0.32  0.057 0.097
```